

Chapitre 3 : Les bases de la syntaxe Python

Oscar Junior WAGOUSSI

ENEAM

February 25, 2025

Objectifs du Chapitre 3

- ▶ Comprendre la déclaration et la manipulation de variables en Python.
- ▶ Découvrir les types de données de base (entiers, flottants, chaînes de caractères) et la conversion de types.
- ▶ Apprendre à utiliser les opérateurs arithmétiques et logiques.
- ▶ Mettre en place des structures conditionnelles (`if`, `elif`, `else`).
- ▶ Maîtriser les boucles `while` et `for`.

Variables

- ▶ **Définition** : Une variable est une sorte de boîte virtuelle dans laquelle on peut stocker une ou plusieurs données.
- ▶ Il est possible d'assigner une variable à une autre.
- ▶ **Opérations** : On peut effectuer des opérations (calculs, concaténations, etc.) avec des variables.
- ▶ **Création en Python** :
 - ▶ Syntaxe : `<nom_de_variable> = <valeur>`
 - ▶ Python utilise l'opérateur `=` pour attribuer une valeur.
 - ▶ L'affectation d'une valeur déclare et initialise la variable ; il n'est pas possible de déclarer une variable sans lui affecter une valeur initiale.

```
>>> age = 30
>>> age
30
```

```
>>> age = 30
>>> age = age + 10
>>> age
40
```

Le nom des variables

- ▶ Un nom de variable doit refléter son contenu, comme une étiquette sur un carton.
- ▶ Facilite la lecture et la maintenance du code.

Recommandations :

- ▶ Utiliser des noms descriptifs dans le code (age, prix_article, etc.).
- ▶ Éviter les abréviations obscures (p_a est moins clair que prix_article).
- ▶ Suivre une convention commune (snake_case en Python).
- ▶ Uniquement des caractères alphanumériques et tirets bas (_). Pas d'accents ni d'espaces. Ne doit pas commencer par un chiffre.
- ▶ Sensibles à la casse : **Age** et **age** sont deux variables différentes.
- ▶ Certains noms sont réservés (if, while, def, etc.) et ne peuvent pas être utilisés comme variables.

Les types de données (1)

Types de base en Python :

- ▶ **int (entiers) :**
 - ▶ Représente des nombres entiers, positifs ou négatifs.
 - ▶ Exemple : 10, -5, 1000
 - ▶ Pas de limite de taille en Python (contrairement à d'autres langages comme C ou Java).
- ▶ **float (flottants) :**
 - ▶ Représente des nombres décimaux.
 - ▶ Exemple : 3.14, -2.7, 1.0e3 (notation scientifique).
- ▶ **bool (booléens) :**
 - ▶ Représente une valeur de vérité.
 - ▶ Deux valeurs possibles : True (vrai) et False (faux).
- ▶ **str (chaîne de caractères) :**
 - ▶ Représente une suite de caractères Unicode.
 - ▶ Exemple : "Bonjour", 'Python'.
 - ▶ La taille d'une chaîne est dynamique (pas de limite fixe).

Les types de données (2)

Structures de données en Python :

- ▶ **list (listes) :**
 - ▶ Collection ordonnée et modifiable d'éléments.
 - ▶ Exemple : `[1, 2, 3]`, `["a", "b", "c"]`.
 - ▶ Peut contenir des types mixtes.
- ▶ **tuple (tuples) :**
 - ▶ Collection ordonnée et immuable.
 - ▶ Exemple : `(1, 2, 3)`, `("a", "b")`.
 - ▶ Plus efficace en mémoire que les listes.
- ▶ **set (ensembles) :**
 - ▶ Collection non ordonnée d'éléments uniques.
 - ▶ Exemple : `{1, 2, 3}`, `{"a", "b"}`.
 - ▶ Ne permet pas de doublons.
- ▶ **dict (dictionnaires) :**
 - ▶ Collection non ordonnée de paires clé-valeur.
 - ▶ Exemple : `{"nom": "Alice", "âge": 20}`.
 - ▶ Permet un accès rapide aux valeurs via leurs clés.

Les types de données (2)

Caractéristiques clés :

- ▶ Python est **dynamiquement typé** : pas besoin de déclarer le type d'une variable.
- ▶ Pour connaître le type d'une variable, utilisez `type(ma_variable)`.
- ▶ Les **annotations de type** permettent d'indiquer un type attendu (`x: int = 10`).

Exemple d'annotation de type

Listing 1: Exemple d'annotation de type

```
1 def addition(a: int, b: int) -> int:  
2     return a + b  
3
```

Dans cet exemple, la fonction `addition` est annotée pour préciser que :

- ▶ Les arguments `a` et `b` doivent être des `int` (entiers).
- ▶ La fonction renvoie un `int`.

Exemple : Déclaration et affichage de variables

Listing 2: Variables de différents types

```
1 # Déclaration de variables
2 age = 20                # int
3 prix_article = 19.99    # float
4 nom = "Alice"           # str
5 est_etudiant = True     # bool
6
7 # Affichage
8 print("Nom :", nom)
9 print("Age :", age)
10 print("Prix :", prix_article)
11 print("Est étudiant :", est_etudiant)
12
13 # Vérifier le type
14 print(type(nom))        # <class 'str'>
15 print(type(prix_article)) # <class 'float'>
16
```

Exemple : Listes et dictionnaires

Listing 3: Exemples de listes et dictionnaires

```
1 # Liste
2 fruits = ["pomme", "banane", "orange"]
3 print("Liste de fruits :", fruits)
4 print("Premier fruit :", fruits[0])
5
6 # Dictionnaire
7 etudiant = {
8     "nom": "Alice",
9     "age": 20,
10    "options": ["math", "informatique"]
11 }
12 print("Dictionnaire étudiant :", etudiant)
13 print("Nom :", etudiant["nom"])
14
```

Exemples d'erreurs de création de variables

Listing 4: Exemples de déclaration de variables invalides

```
1 # 1) Ne peut pas commencer par un chiffre
2 1x = 10
3 # Erreur typique : "SyntaxError: invalid syntax"
4 # 2) Caractères spéciaux non autorisés
5 x-y = 20
6 # Erreur typique : "SyntaxError: can't assign to operator"
7 # 3) Mots-clés réservés (ex. 'import', 'if', etc.)
8 import = 30
9 if = 40
10 # Erreur typique : "SyntaxError: invalid syntax"
11 # 4) Espaces ou accents dans le nom
12 nom élève = "Bob"
13 déjà = 50
14 # Erreur typique : "SyntaxError: invalid syntax"
15 # 5) Caractère non alphanumérique ou underscore
16 x! = 60
17 # Erreur typique : "SyntaxError: invalid syntax"
18
```

Déclaration multiple de variables en Python

Python permet d'affecter plusieurs variables en une seule ligne :

Listing 5: Exemples de déclaration de variables invalides

```
1 # Affectation multiple
2 a, b, c = 1, 2, 3
3
4 # Affichage des valeurs
5 print("a =", a)
6 print("b =", b)
7 print("c =", c)
8
```

Avantages :

- ▶ Permet d'assigner plusieurs valeurs en une seule ligne.
- ▶ Rend le code plus lisible et concis.
- ▶ Évite les répétitions inutiles.

Exercices : Variables et types

- ▶ **Exercice 1** : Créez une variable pour chaque type suivant : `int`, `float`, `str`, `bool`. Affichez-les et vérifiez leur type avec `type()`.
- ▶ **Exercice 2** : Demandez à l'utilisateur son nom et son âge, puis affichez un message de bienvenue en utilisant ces informations.

Variables et conversions de types

- **Conversion de types** : `int()`, `float()`, `str()` permettent de changer le type.

Exemple : Variables et conversions

Listing 6: Variables et conversions de types

```
1 # Affectation
2 age = 20          # int
3 taille = 1.75     # float
4 nom = "Alice"     # str
5
6 # Conversion
7 age_str = str(age)      # "20"
8 taille_int = int(taille) # 1 (tronqué)
9
10 print("Nom :", nom)
11 print("Age :", age, "ans")
12 print("Taille :", taille, "m")
13 print("Taille entière :", taille_int)
14
```

Opérateurs arithmétiques et logiques

Opérateurs arithmétiques

- ▶ `+`, `-`, `*`, `/` : addition, soustraction, multiplication, division
- ▶ `//` : division entière
- ▶ `%` : modulo (reste de la division)
- ▶ `**` : puissance (ex. `2**3 = 8`)

Opérateurs logiques

- ▶ `and`, `or`, `not`
- ▶ `==`, `!=`, `<`, `>`, `<=`, `>=`
- ▶ Ex. `(x > 5) and (y < 10)`

Exemple : Opérateurs

Listing 7: Opérateurs arithmétiques et logiques

```
1 x = 10
2 y = 3
3
4 # Arithmétique
5 print("x + y =", x + y)      # 13
6 print("x / y =", x / y)      # 3.3333...
7 print("x // y =", x // y)    # 3
8 print("x % y =", x % y)      # 1
9 print("x ** y =", x ** y)    # 1000
10
11 # Logique
12 print("x > 5 and y < 5 :", (x > 5) and (y < 5))  # True
13 print("x == 10 or y == 10 :", (x == 10) or (y == 10)) # True
14
```

Indentation de bloc (1)

- ▶ Python utilise l'indentation pour définir les constructions de contrôle et de boucle.
- ▶ L'indentation améliore la lisibilité mais nécessite une attention particulière à l'utilisation des espaces.
- ▶ Une indentation incorrecte peut entraîner un comportement inattendu ou une `IndentationError`.
- ▶ Python utilise le symbole deux-points ":" pour indiquer le début d'un bloc de code.
- ▶ Contrairement à d'autres langages, Python ne possède pas de délimiteurs explicites comme "{}" pour indiquer la fin d'un bloc.

Indentation de bloc (2)

Exemple 1 : Fonction avec indentation correcte

Listing 8: Fonction avec indentation correcte

```
def my_function():  
    a = 2  
    return a  
  
print(my_function()) # Hors du bloc de la fonction
```

Exemple 2 : Condition avec indentation correcte

Listing 9: Condition avec indentation correcte

```
if a > b:  
    print(a) # Fait partie du bloc if  
else:  
    print(b) # Fait partie du bloc else
```

Indentation de bloc (3)

- Un bloc contenant une seule instruction peut être écrit sur une seule ligne.

Listing 10: Bloc sur une seule ligne

```
if a > b: print(a)
else: print(b)
```

- Cependant, essayer de le faire avec plusieurs instructions ne fonctionne pas :

Listing 11: Erreurs d'indentation

```
if x > y: y = x
        print(y)  # Erreur d'indentation

if x > y: while y != z: y -= 1  # SyntaxError
```

Indentation de bloc (4)

Blocs vides :

- ▶ Un bloc vide provoque une `IndentationError`. Pour éviter cela, utilisez 'pass' :

```
def will_be_implemented():  
    pass
```

Espaces ou tabulations :

- ▶ Toujours utiliser 4 espaces par niveau d'indentation (selon la PEP 8).
- ▶ L'utilisation exclusive de tabulations est possible, mais les espaces sont préférés.

Optimisation de l'Indentation en Python

Problème : Code trop imbriqué (difficile à lire)

Listing 12: Code trop imbriqué

```
1 age = int(input("Entrez votre age: "))
2
3 if age >= 0:
4     if age < 18:
5         if age < 12:
6             print("Enfant")
7         else:
8             print("Adolescent")
9     else:
10        print("Adulte")
11 else:
12     print("Âge invalide")
13
```

Optimisation de l'Indentation en Python

Solution : Réduire l'imbrication avec 'elif'

Listing 13: Version optimisée

```
1 age = int(input("Entrez votre age: "))
2
3 if age < 0:
4     print("Âge invalide")
5 elif age < 12:
6     print("Enfant")
7 elif age < 18:
8     print("Adolescent")
9 else:
10    print("Adulte")
11
```

*L'utilisation de **elif** permet d'éviter l'imbrication excessive, rendant le code plus lisible et plus facile à comprendre.*

Structures conditionnelles

- ▶ `if` : exécute un bloc si la condition est vraie.
- ▶ `elif` : alternative si la condition `if` est fausse.
- ▶ `else` : exécute un bloc si toutes les conditions précédentes sont fausses.

Syntaxe générale :

```
if condition1:  
    bloc d'instructions  
elif condition2:  
    bloc d'instructions  
else:  
    bloc d'instructions
```


Exemple : if, elif, else

Listing 14: Conditions en Python

```
1 age = int(input("Entrez votre age : "))
2
3 if age < 0:
4     print("Erreur : l'age ne peut pas être négatif.")
5 elif age < 18:
6     print("Vous êtes mineur.")
7 else:
8     print("Vous êtes majeur.")
9
```

Les boucles en Python

- ▶ **Boucle while** : répète un bloc d'instructions tant qu'une condition est vraie.

```
while condition:  
    bloc d'instructions
```

- ▶ **Boucle for** : itère sur les éléments d'une séquence (liste, chaîne, etc.) ou sur une plage de valeurs.

```
for variable in sequence:  
    bloc d'instructions
```

Exemple : Boucle while

Listing 15: Exemple d'une boucle while

```
1  compteur = 0
2
3  while compteur < 5:
4      print("Compteur =", compteur)
5      compteur += 1  # Incrémentation
6      print("Fin de la boucle while")
7
```

Exemple : Boucle for

Listing 16: Exemple d'une boucle for

```
1 # Boucle sur une plage de valeurs
2 for i in range(5):
3     print("i =", i)
4
5 # Boucle sur une liste
6 fruits = ["pomme", "banane", "orange"]
7 for fruit in fruits:
8     print("Fruit :", fruit)
9
```

Utilisation de range, break et match en Python

Range

Il est possible de créer une boucle facilement avec range :

Listing 17: Exemple d'une boucle for

```
1 for i in range(0, 100):  
2     print(i)  
3
```

Utilisation de range, break et match en Python

Stopper une boucle avec break

Pour stopper immédiatement une boucle, on peut utiliser le mot-clé 'break' :

Listing 18: Exemple d'une boucle for

```
1 liste = [1, 5, 10, 15, 20, 25]
2
3 for i in liste:
4     if i > 15:
5         print("On stoppe la boucle")
6         break
7     print(i)
8
```

Utilisation de range, break et match en Python

Match

L'instruction 'match' confronte la valeur d'une expression à plusieurs filtres successifs donnés par les instructions 'case' :

Listing 19: Exemple d'une boucle for

```
1 commande = "pizza"
2
3 match commande:
4     case "pizza":
5         print("Commande de pizza enregistrée")
6     case "burger":
7         print("Commande de burger enregistrée")
8     case _:
9         print("Commande inconnue")
10
```

Exercices pratiques

- ▶ **Exercice 3** : Écrire un programme qui demande un nombre à l'utilisateur et indique s'il est pair ou impair (utilisez l'opérateur modulo %).
- ▶ **Exercice 4** : Écrire un programme qui affiche la table de multiplication (de 1 à 10) pour un nombre donné.

Conclusion du Chapitre 3

- ▶ Vous avez découvert la syntaxe de base de Python : variables, types, opérateurs.
- ▶ Vous maîtrisez désormais les structures conditionnelles (`if`, `elif`, `else`) et les boucles (`while`, `for`).
- ▶ Continuez à pratiquer ces notions à travers des exercices et des mini-projets pour vous familiariser avec la logique de programmation.